

NATIONAL UNIVERSITY OF MODERN LANGUAGES

Assignment # 03



SUBMITTED BY:

ZULQARNAIN HASSAN

SUBMITTED TO:

SIR. M WAQAR

Subject: data structure & algorithm

Roll No: SP21379

Department: Software engineering

Section: BSSE-III-A-afternoon

Answer N0 1

Write a function that can show the minimum and maximum elements in a binary search tree and update the value of a minimum element with some random value.

```
#include<iostream>
using namespace std;
struct node{
int data;
node* left;
node* right;
};
node* root=NULL;
void show(node*);
node *insert(node *,int);
main(){
    root=insert(root,6);
    root=insert(root,4);
    root=insert(root,5);
    root=insert(root,3);
    root=insert(root,10);
    root=insert(root,8);
    root=insert(root,11);
    show(root);
}
node *insert(node *root,int n)
{
    if (root == NULL)
    {
        node *root1 = new node;
        root1->data = n;
        root1->left = NULL;
        root1->right = NULL;
        return root1;
    }

    else if (n < root->data)
    {
        root->left = insert(root->left,n);
    }
    else if (n > root->data)
    {
        root->right = insert(root->right, n);
    }
}
```

```

}
void show(node* root)
{
    node* root1=root;
    while(root1->left!=NULL)
    {
        root1=root1->left;
    }
    while(root->right!=NULL)
    {
        root=root->right;
    }
    cout<<"\nthe minimum number in the binary search tree is: "<<root1->data;
    cout<<"\n\nthe maximum number in the binary search tree is: "<<root->data;
    root1->data=1;
    cout<<"\n\nthe random updated value of the minimum number in the binary search tree
is: "<<root1->data<<endl;

}

```

```

PS E:\C++> cd "e:\C++\" ; if ($?) { g++ min_max.cpp -o min_max } ; if ($?) { .\min_max }

the minimum number in the binary search tree is: 3

the maximum number in the binary search tree is: 11

the random updated value of minimum number in the binary search tree is: 1
PS E:\C++>

```

Answer N0 2

Suppose we have a binary search tree having integer number as its keys, write a function that calculate sum of all values of nodes in a binary search tree.

```

#include <iostream>
using namespace std;
struct node
{
    int key;
    node *left;
    node *right;
};
node *root = NULL;
int sum(node *);
node *insert(node *, int);
main()
{
    root = insert(root, 6);
}

```

```

root = insert(root, 4);
root = insert(root, 5);
root = insert(root, 3);
root = insert(root, 10);
root = insert(root, 8);
root = insert(root, 11);
int sum_of_values=sum(root);
cout<<"\n\nsum of all values of nodes in a binary search tree:
"<<sum_of_values<<endl;
}
node *insert(node *root, int n)
{
    if (root == NULL)
    {
        node *root1 = new node;
        root1->key = n;
        root1->left = NULL;
        root1->right = NULL;
        return root1;
    }

    else if (n < root->key)
    {
        root->left = insert(root->left, n);
    }
    else if (n > root->key)
    {
        root->right = insert(root->right, n);
    }
}
int sum(node *root)
{
    if (root == NULL)
    {
        return 0;
    }

    return root->key + sum(root->left) + sum(root->right);
}

```

```
PS E:\C++> cd "e:\C++\" ; if ($?) { g++ Q2.cpp -o Q2 } ; if ($?) { .\Q2 }

sum of all values of nodes in a binary search tree: 47
PS E:\C++>
```

Answer N0 3

Write a function that can only show leaf nodes present in a binary search tree.

```
#include <iostream>
using namespace std;
struct node
{
    int key;
    node *left;
    node *right;
};
node *root = NULL;
void printLeafNodes(node *);
node *insert(node *, int);
main()
{
    root = insert(root, 6);
    root = insert(root, 4);
    root = insert(root, 5);
    root = insert(root, 3);
    root = insert(root, 10);
    root = insert(root, 8);
    root = insert(root, 11);
    cout<<"leaf nodes are\n";
    printLeafNodes(root);
}

node *insert(node *root, int n)
{
    if (root == NULL)
    {
        node *root1 = new node;
        root1->key = n;
        root1->left = NULL;
        root1->right = NULL;
        return root1;
    }

    else if (n < root->key)
```

```

    {
        root->left = insert(root->left, n);
    }
    else if (n > root->key)
    {
        root->right = insert(root->right, n);
    }
}

void printLeafNodes(node* root) {
    if (root == nullptr)
        return;

    if (root->left == nullptr && root->right == nullptr)
        std::cout << root->key << std::endl;

    printLeafNodes(root->left);
    printLeafNodes(root->right);
}

```

```

PS E:\C++> cd "e:\C++\" ; if ($?) { g++ Q3.cpp -o Q3 } ; if ($?) { .\Q3
leaf nodes are
3
5
8
11
PS E:\C++>

```